

05506026 Digital Image Processing (Java)

Lab 3

Objectives

1. Understand how to perform some simple intensity transformations
2. Understand how to calculate the histogram of an image
3. Understand how to calculate the contrast of an image
4. Understand how to adjust the contrast of an image

Brightness

Brightness is one of attributes of an image. To control the brightness of an image, we need to access to each pixel of the image and add the desired brightness to every colour channel in the pixel. Note that the value of each channel must be clamped within the value of bit depth. For example, the value in an 8-bit colour channel should not be outside the range [0, 255] after adjusting the brightness.

The code to adjust the brightness is as follow.

```

ImageManager.java
class ImageManager
{
    ...
    public void adjustBrightness(int brightness)
    {
        if (img == null)    return;

        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                int color = img.getRGB(x, y);
                int r = (color >> 16) & 0xff;
                int g = (color >> 8) & 0xff;
                int b = color & 0xff;

                r = r + brightness;
                r = r > 255? 255: r;
                r = r < 0? 0: r;

                g = g + brightness;
                g = g > 255? 255: g;
                g = g < 0? 0: g;

                b = b + brightness;
                b = b > 255? 255: b;
                b = b < 0? 0: b;

                color = (r << 16) | (g << 8) | b;

                img.setRGB(x, y, color);
            }
        }
    }
    ...
}

```

Below are the images of Mandril which lower 100 of the brightness and raise 100 of the brightness respectively.



Negative image

Negative image or invert colour can be done by perform this equation to every colour channel of each pixel in the image.

$$s = L - r$$

Where L is the bit depth of the colour channel. For example, an 8-bit colour channel will have L as 255 ($2^8 - 1$).

The code to produce the negative image is as follow.

```

ImageManager.java
class ImageManager
{
    ...
    public void invert()
    {
        if (img == null)    return;

        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                int color = img.getRGB(x, y);
                int r = (color >> 16) & 0xff;
                int g = (color >> 8) & 0xff;
                int b = color & 0xff;

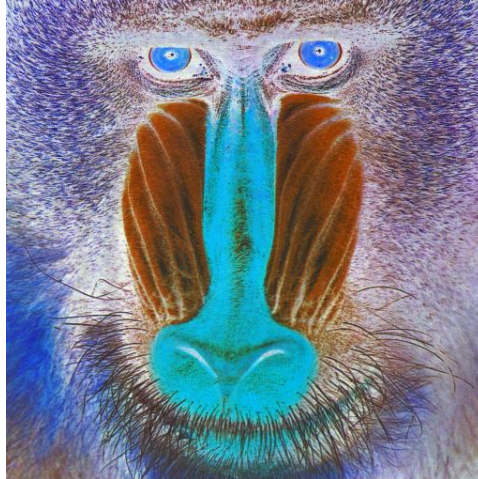
                r = 255 - r;
                g = 255 - g;
                b = 255 - b;

                color = (r << 16) | (g << 8) | b;

                img.setRGB(x, y, color);
            }
        }
        ...
    }
}

```

Below is the negative image of Mandril.



Histogram

Histogram is a representation of the distribution of intensity data in a digital image. To get the histogram of an image, we need to iterate through the image and count the number of pixels of each intensity value. Note that at this level, we will focus on only histogram of grayscale. Therefore, we need to convert the colour image to grayscale first before counting the values.

Histogram is useful for many algorithms after this. For example, contrast, thresholding, edge detection, etc.

The code to count the histogram is as below.

```
ImageManager.java
class ImageManager
{
    ...
    public int[] getGrayscaleHistogram()
    {
        if (img == null)    return null;

        convertToGrayscale();

        int[] histogram = new int[256];

        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                int color = img.getRGB(x, y);
                int gray = color & 0xff;

                histogram[gray]++;
            }
        }

        restoreToOriginal();

        return histogram;
    }
    ...
}
```

The histogram can be written into a CSV file using the code below.

```

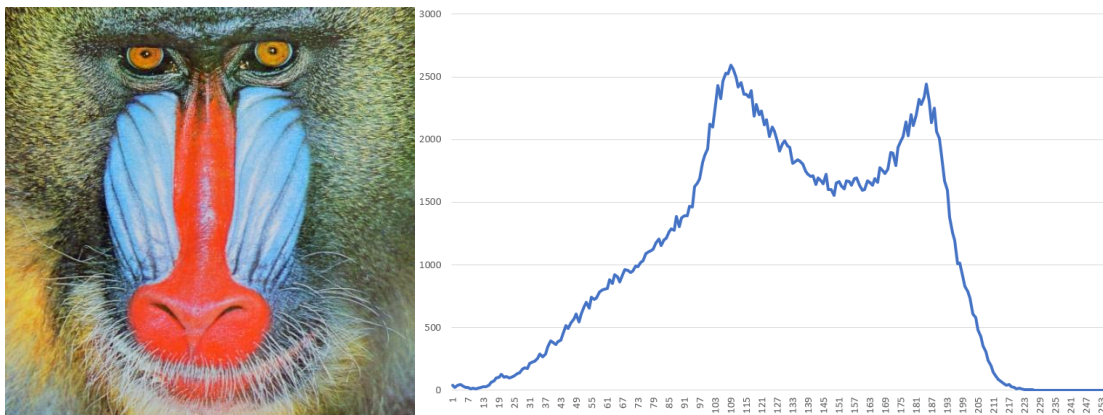
ImageManager.java
class ImageManager
{
    ...
    public void writeHistogramToCSV(int[] histogram, String fileName)
    {
        try
        {
            FileWriter fw = new FileWriter(fileName);

            for (int i = 0; i < histogram.length; i++)
            {
                fw.write(histogram[i] + ",");
            }

            fw.close();
        }
        catch (IOException e)
        {
            System.out.println(e);
        }
    }
    ...
}

```

Below is the histogram of Mandril.bmp



Contrast

Contrast is the difference in intensity values of a digital image, calculated by the standard deviation of luminance value of pixels (calling Root Mean Square or RMS Contrast)

$$\sqrt{\frac{1}{MN} \sum_{i=0}^{N-1} \sum_{j=0}^{M-1} (I_{ij} - \bar{I})^2}$$

Where luminance value (I_{ij}) is

$$Y_{\text{linear}} = 0.2126R_{\text{linear}} + 0.7152G_{\text{linear}} + 0.0722B_{\text{linear}}$$

The code to calculate the contrast of an image is as follow.


```

ImageManager.java
class ImageManager
{
    ...
    public float getContrast()
    {
        if (img == null)    return 0;

        float contrast = 0;

        int[] histogram = getGrayscaleHistogram();
        float avgIntensity = 0;
        float pixelNum = width * height;

        for (int i = 0; i < histogram.length; i++)
        {
            avgIntensity += histogram[i] * i;
        }

        avgIntensity /= pixelNum;

        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                int color = img.getRGB(x, y);
                int value = color & 0xff;

                contrast += Math.pow((value) - avgIntensity, 2);
            }
        }

        contrast = (float)Math.sqrt(contrast / pixelNum);

        return contrast;
    }
    ...
}

```

Contrast can be adjusted by changing the range of histogram of the image. For example, increasing the contrast for 100 means that the histogram of the image will be stretched by 100. Meaning that the standard deviation will also be increased in both directions, 50 to the left and 50 to the right.

Therefore, each colour channel will be calculated with the new minimum and maximum values using the formula below.

$$I_N = (I - \text{Min}) \frac{\text{newMax} - \text{newMin}}{\text{Max} - \text{Min}} + \text{newMin}$$

The code to calculate the new contrast of an image is as follow.

```

ImageManager.java
class ImageManager
{
    ...
    public void adjustContrast(int contrast)
    {
        if (img == null)    return;

        float currentContrast = getContrast();

        int[] histogram = getGrayscaleHistogram();
        float avgIntensity = 0;
        float pixelNum = width * height;

        for (int i = 0; i < histogram.length; i++)
        {
            avgIntensity += histogram[i] * i;
        }

        avgIntensity /= pixelNum;

        float min = avgIntensity - currentContrast;
        float max = avgIntensity + currentContrast;

        float newMin = avgIntensity - currentContrast - contrast / 2;
        float newMax = avgIntensity + currentContrast + contrast / 2;

        newMin = newMin < 0? 0: newMin;
        newMax = newMax < 0? 0: newMax;
        newMin = newMin > 255? 255: newMin;
        newMax = newMax > 255? 255: newMax;

        if (newMin > newMax)
        {
            float temp = newMax;
            newMax = newMin;
            newMin = temp;
        }

        float contrastFactor = (newMax - newMin) / (max - min);
    }
}

```

```

for (int y = 0; y < height; y++)
{
    for (int x = 0; x < width; x++)
    {
        int color = img.getRGB(x, y);
        int r = (color >> 16) & 0xff;
        int g = (color >> 8) & 0xff;
        int b = color & 0xff;

        r = (int)((r - min) * contrastFactor + newMin);
        r = r > 255? 255: r;
        r = r < 0? 0: r;

        g = (int)((g - min) * contrastFactor + newMin);
        g = g > 255? 255: g;
        g = g < 0? 0: g;

        b = (int)((b - min) * contrastFactor + newMin);
        b = b > 255? 255: b;
        b = b < 0? 0: b;

        color = (r << 16) | (g << 8) | b;

        img.setRGB(x, y, color);
    }
}
...
}

```

Quest 004

Power-law transformation, or Gamma transformation, is an intensity transformation which adjusts the gamma level of the image by applying the following equation to each colour channel.

$$s = c \times r^\gamma$$

Where c is a constant and γ is the desired gamma level. Normally, input colour channel must be normalised (converted into the range of $[0, 1]$) first before taking the power of γ . Also, c is the constant which indicates the bit depth (255 for 8-bit colour channel). Note that the value of each

channel must be clamped within the value of bit depth. For example, the value in an 8-bit colour channel should not be outside the range [0, 255] after adjusting the gamma level.

Implement the method of power-law transformation and upload two Mandril images which adjust gamma to 2.2 and 0.4.

The player who finishes this task will be awarded **15 EXP**.

Deadline: 7 September 2025

Quest 005

Implement the method of contrast adjustment and upload two Mandril images which adjust contrast to 100 and -100.

The player who finishes this task will be awarded **10 EXP** and **1 Move**.

Deadline: 7 September 2025